K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

# Lab Assignment-1

School of Engineering and Technology

Name = Tamanna

RollNo. = 2401201138

Program =BCA (AI & DS)

# Operating System Lab ENCA252

Submitted to :- Jyoti Mam

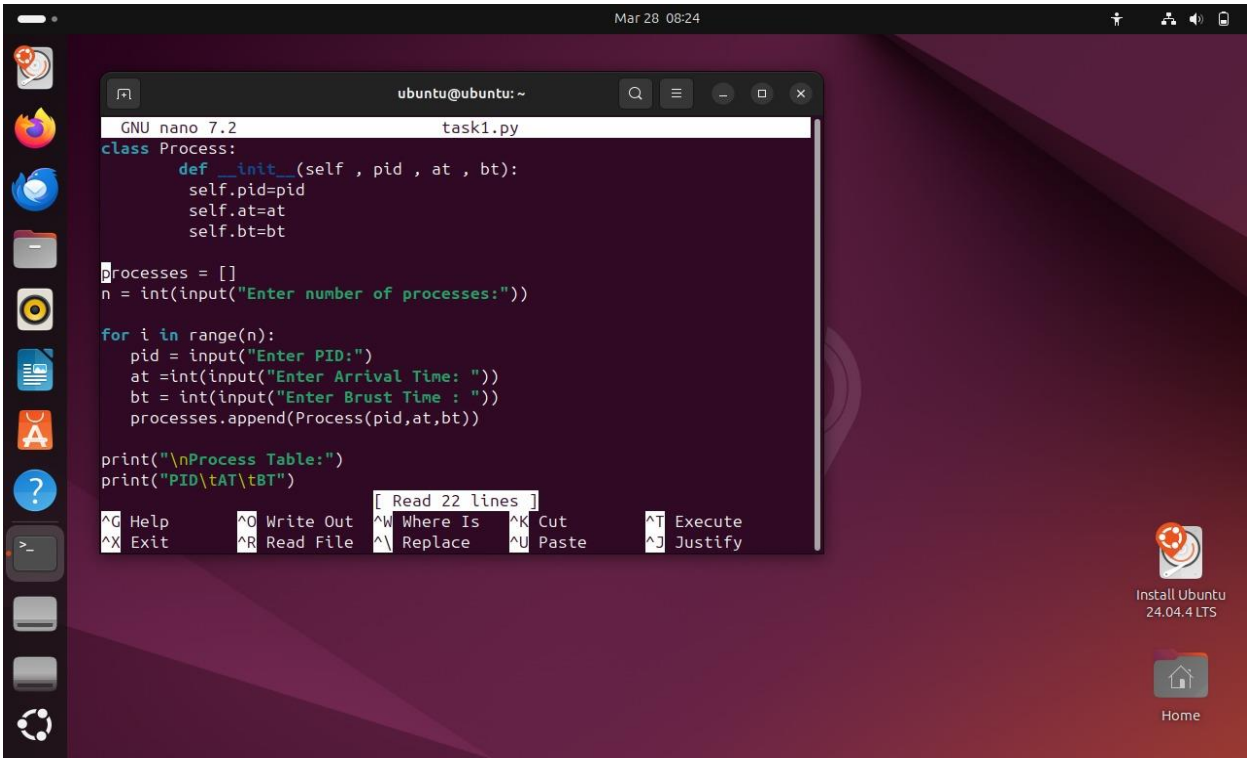
# Implementation and Analysis of CPU Scheduling Algorithms (FCFS & SJF)

## Problem Statement

CPU Scheduling is a core function of an operating system that determines the order in which processes access the CPU. In this assignment, students will implement FCFS and SJF scheduling algorithms and evaluate their performance using standard metrics.

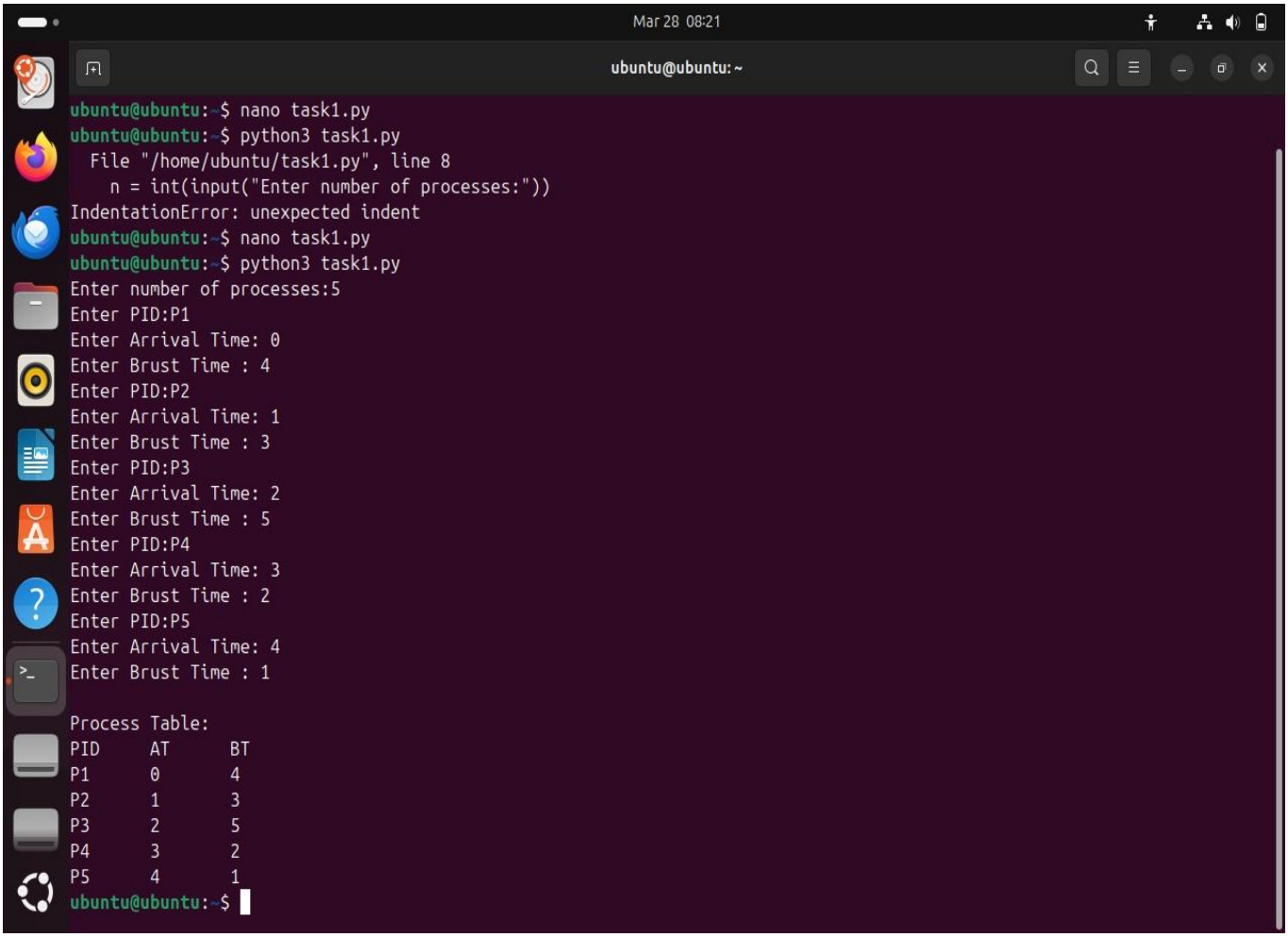
## Tools / Technology Used

Python 3.12.3, Linux OS (Ubuntu 24.04 LTS via VirtualBox), nano/gedit text editor



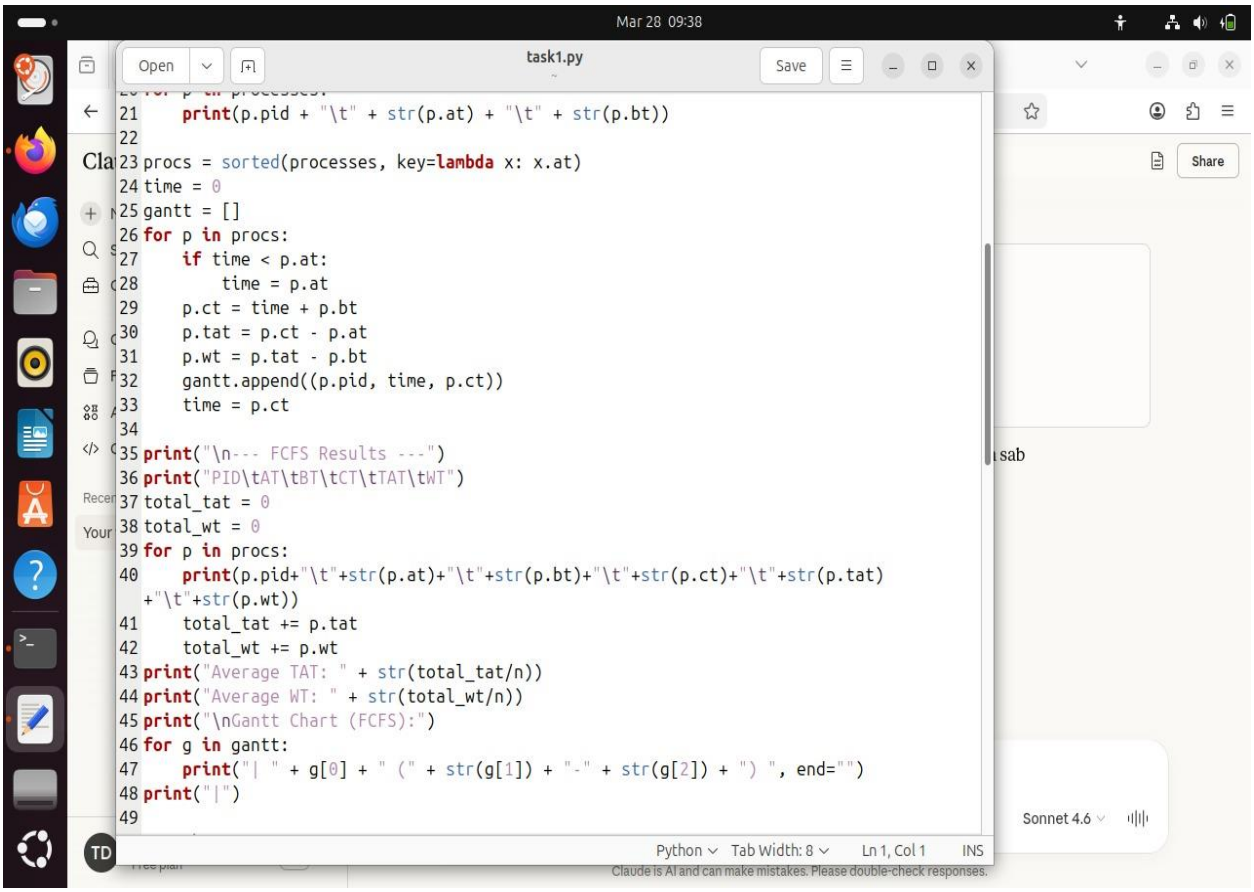

**Task 1 - Process Class and Input Handling Code (task1.py)**

A Process class is defined with attributes PID, Arrival Time and Burst Time. Input is taken for each process and stored in a list for further processing



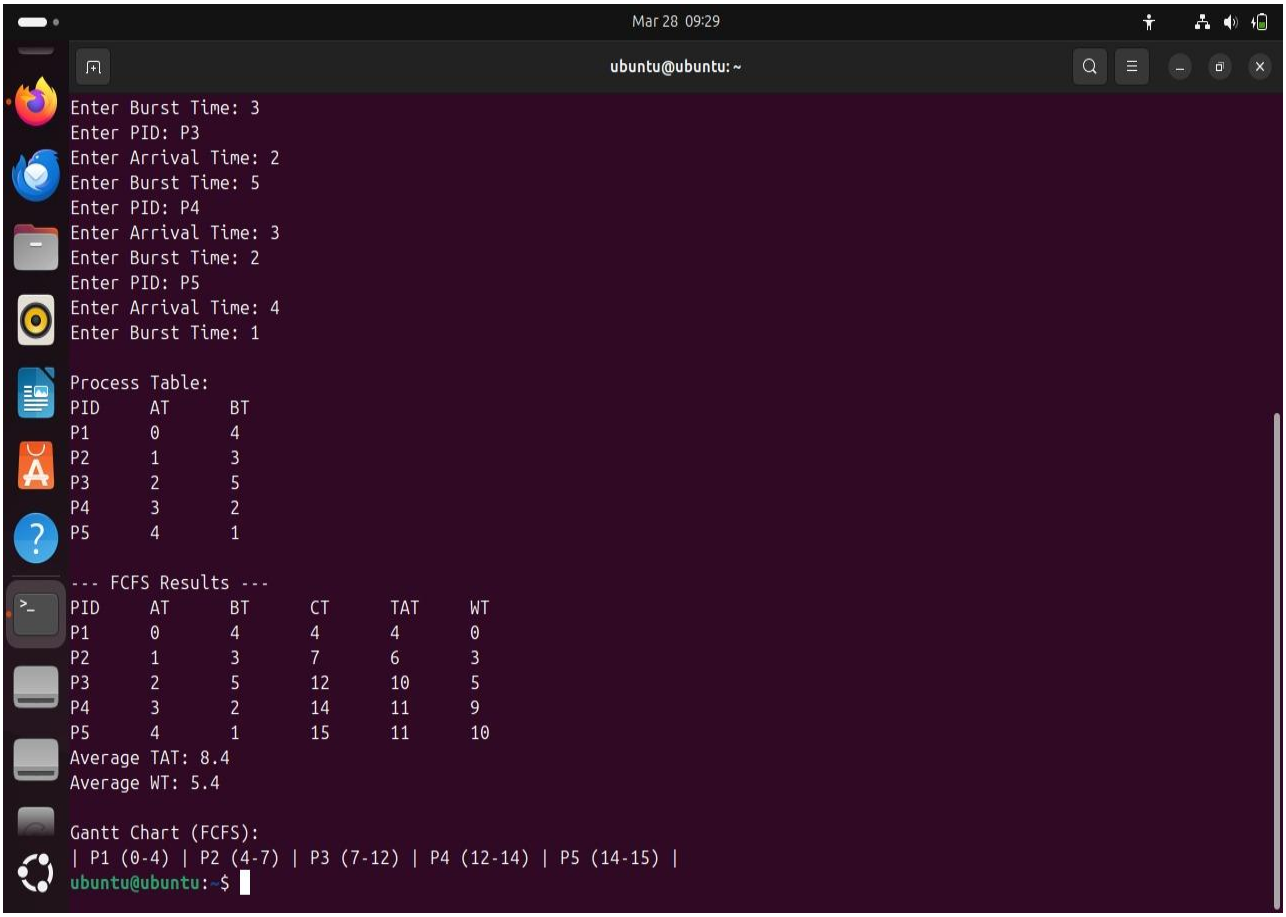

Task 1 - Process Table Output

5 processes were entered with their Arrival Time and Burst Time. The program successfully displays the Process Table showing PID, AT and BT for all processes.



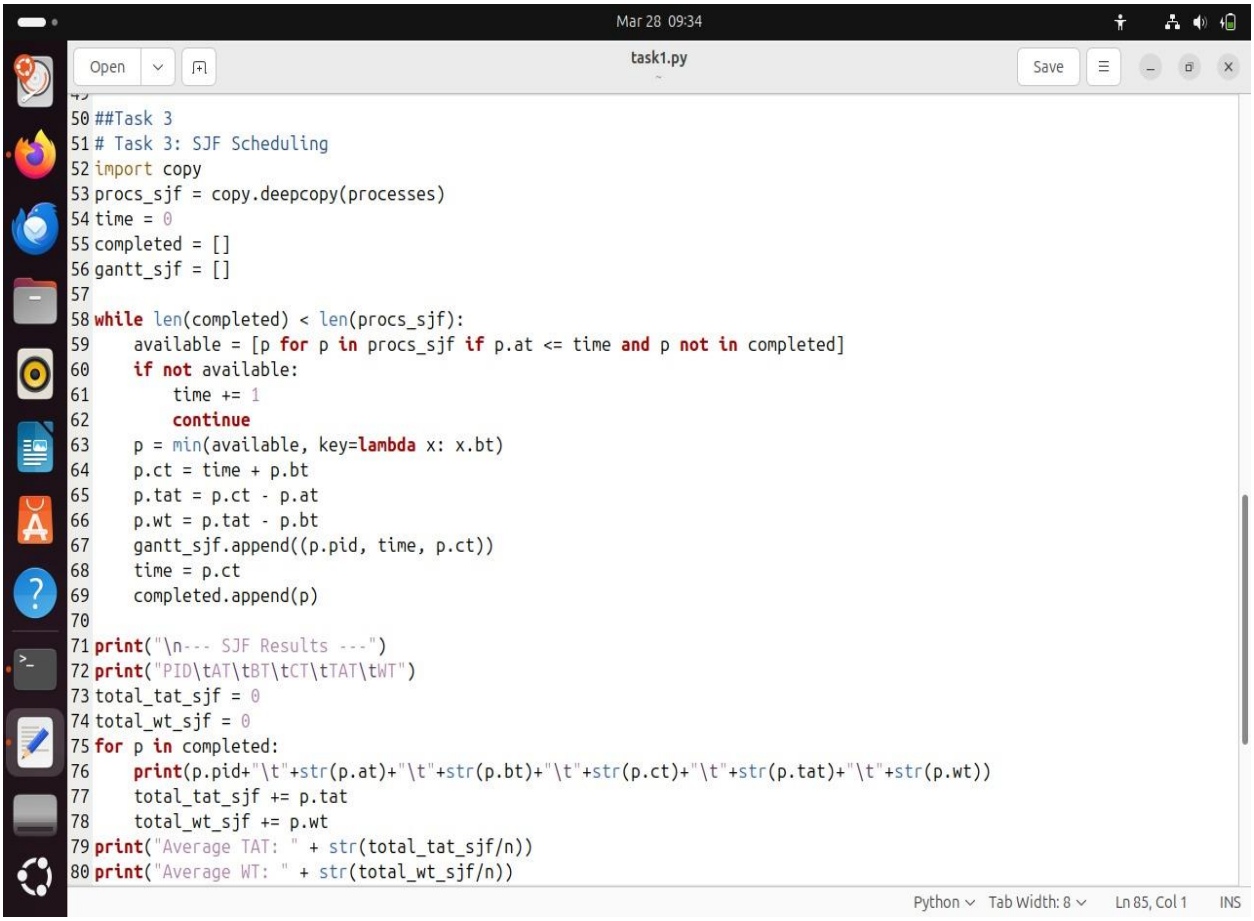

Task 2 - FCFS Scheduling Algorithm Code

Processes are sorted by Arrival Time. Completion Time, Turnaround Time and Waiting Time are calculated for each process. Average TAT and WT are computed and Gantt Chart is displayed.



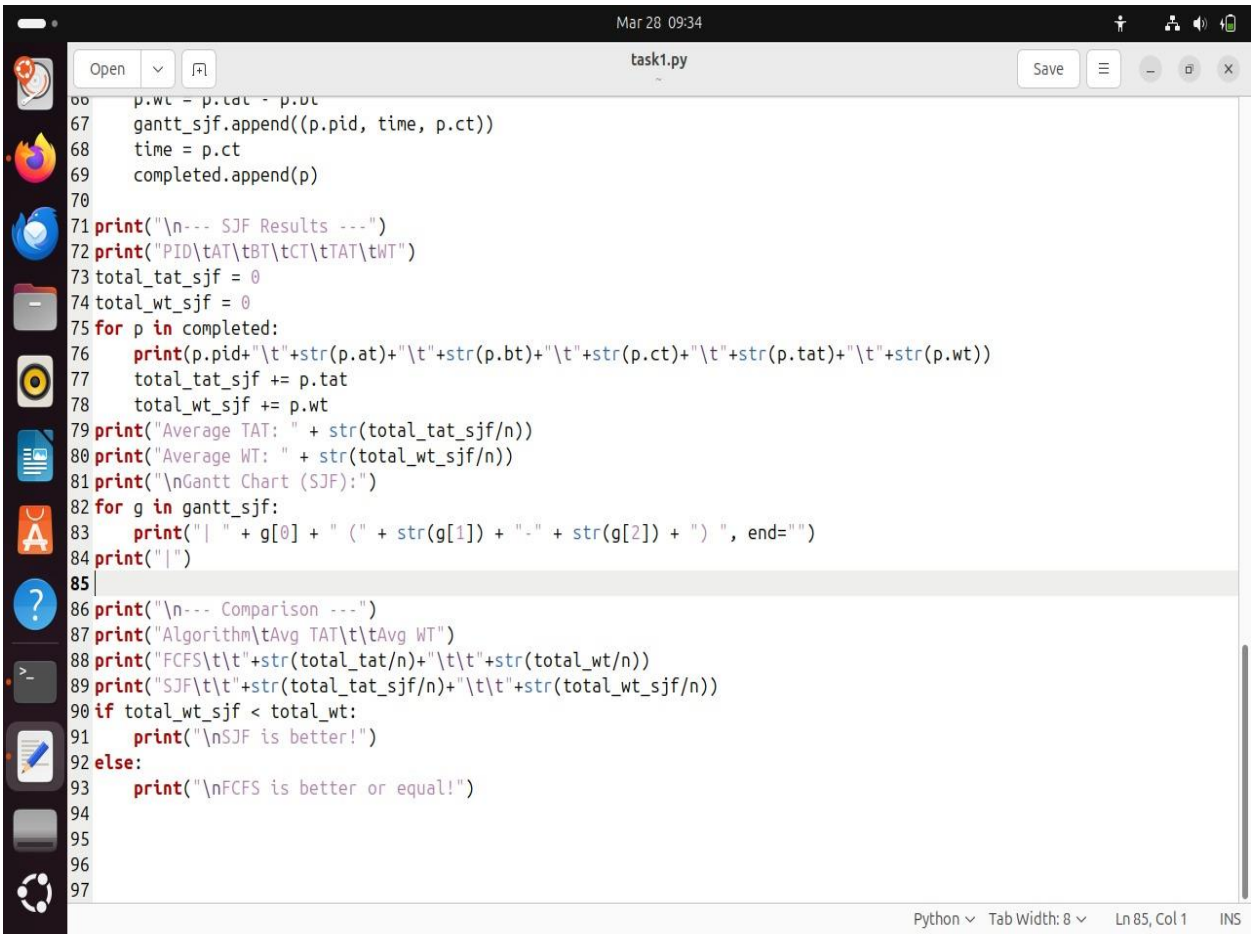

## Task 2 - FCFS Scheduling Output

FCFS algorithm executed all 5 processes in order of Arrival Time. Completion Time, Turnaround Time and Waiting Time were calculated for each process. Average TAT is 8.4 and Average WT is 5.4. Gantt Chart shows execution order: P1→P2→P3→P4→P5.



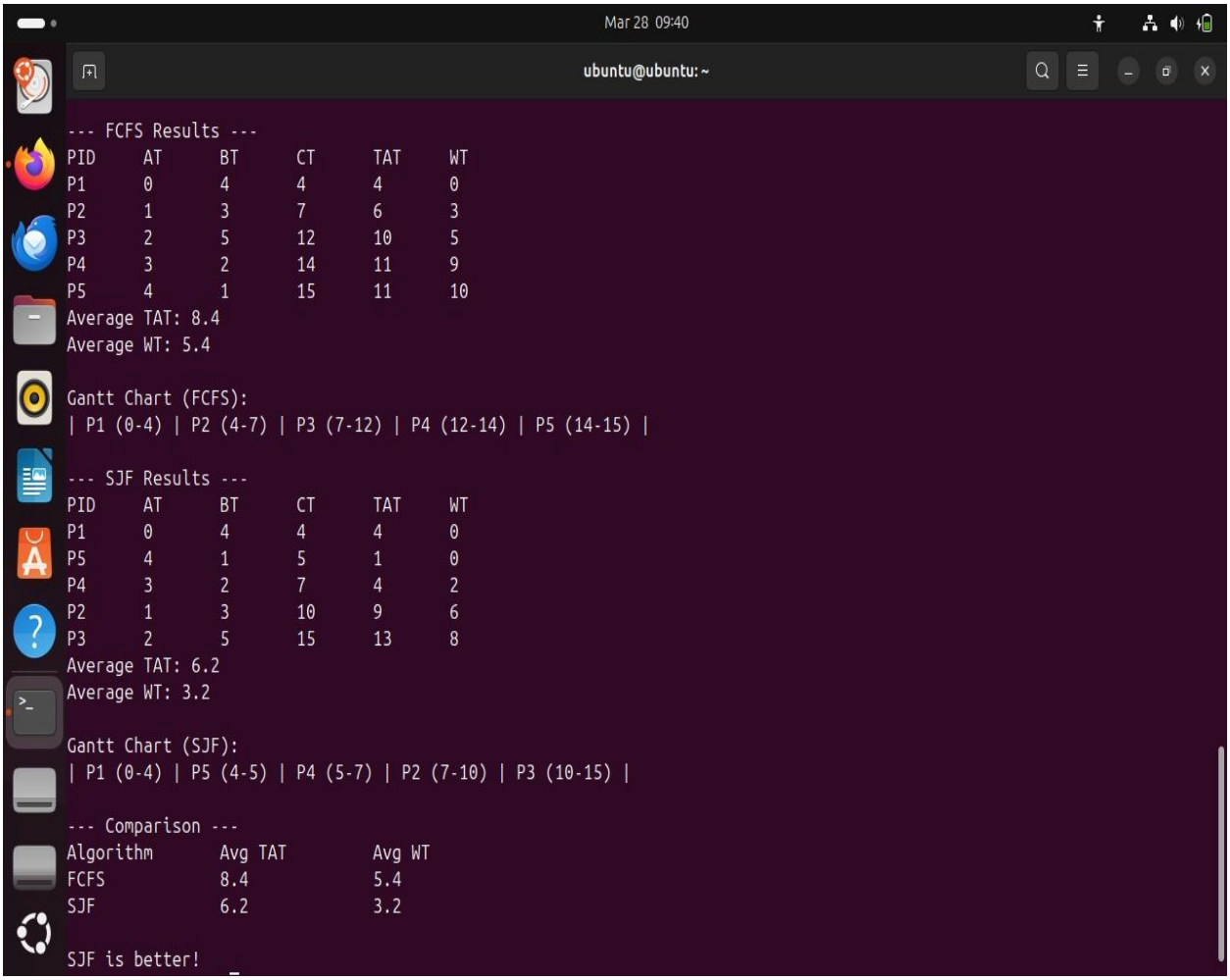

## Task 3 - SJF Scheduling Algorithm Code

SJF algorithm selects the process with shortest Burst Time from available processes. deepcopy is used to preserve original data. The while loop continues until all processes are completed and builds the safe execution sequence




Task 3, 4 & 5 - SJF Results, Gantt Chart and Comparison Code

SJF results are displayed with CT, TAT and WT for each process. Gantt Chart shows SJF execution order. Finally FCFS and SJF are compared based on Average TAT and Average WT to determine which algorithm performs better



```
--- FCFS Results ---
PID     AT      BT      CT      TAT     WT
P1      0       4       4       4       0
P2      1       3       7       6       3
P3      2       5       12      10      5
P4      3       2       14      11      9
P5      4       1       15      11      10
Average TAT: 8.4
Average WT: 5.4

Gantt Chart (FCFS):
| P1 (0-4) | P2 (4-7) | P3 (7-12) | P4 (12-14) | P5 (14-15) |

--- SJF Results ---
PID     AT      BT      CT      TAT     WT
P1      0       4       4       4       0
P5      4       1       5       1       0
P4      3       2       7       4       2
P2      1       3       10      9       6
P3      2       5       15      13      8
Average TAT: 6.2
Average WT: 3.2

Gantt Chart (SJF):
| P1 (0-4) | P5 (4-5) | P4 (5-7) | P2 (7-10) | P3 (10-15) |

--- Comparison ---
Algorithm       Avg TAT         Avg WT
FCFS            8.4             5.4
SJF             6.2             3.2

SJF is better!
```

## Task 3, 4 & 5 - SJF Output, Gantt Chart and Final Comparison

SJF algorithm gives Average TAT of 6.2 and Average WT of 3.2, which is better than FCFS (TAT: 8.4, WT: 5.4). SJF Gantt Chart shows optimized execution order. Conclusion: SJF is better than FCFS as it minimizes waiting time